DISTRIBUTED SYSTEMS & SEARCH ARCHITECTURE WHITEPAPER

NEUTRON v3.0

Distributed Web Search Engine, Server-Streaming JIT Crawler, and Grounded Cited Research Platform

Architectural Thesis: Neutron demonstrates how a production-grade, zero-telemetry search engine with sub-50 ms indexed retrieval, live Chrome-impersonated web acquisition, server-streaming progress telemetry, and optional LLM citation synthesis can operate cohesively across decoupled edge and ConnectRPC microservices within a **$0/month serverless cloud budget**.

CLOUD FOOTPRINT	RPC TOPOLOGY	RETRIEVAL ENGINE	PRIVACY MODEL
$0 / mo	**3 Services**	**FTS5 + JIT**	**Zero Logs**
Render Free + GitHub + Turso	ConnectRPC / Protobuf (`proto3`)	Porter BM25 + 6 Live Adapters	No Cookies, No IP Persistence

PRODUCTION TECHNOLOGY STACK



Author: Md Fahim Sarker

Release: v3.0 (Main)

Repository: https://github.com/fahiiim/Neutron-Web-Crawler

Gateway: https://neutron-gateway.onrender.com

Contents

1 Executive Summary & Engineering Design Rationale 1

2 Cloud Architecture & Deployment Topology 2

2.1 Render Blueprint & Cold-Start Resilience Engineering . . . 3

3 High-Level Design (HLD): Hybrid Cache-First & JIT Streaming 3

3.1 Gateway Caching, Refresh Semantics & Error Resilience . . . 4

4 Low-Level Design (LLD): Internal Mechanics & Algorithms 4

4.1 Contract-First ConnectRPC, Auth Interceptor & 64-Bit IDs . . . 4

4.2 Query Normalization, Stemmed Quorum FTS5 & Concept Re-Ranking . . . 5

4.3 Discovery Scheduling, Chrome TLS Fetching & Deduplication . . . 5

5 Storage Schema & Data Modeling (`Turso LibSQL FTS5`) 7

6 Latency Budgets, Free-Tier Engineering & Verification 7

6.1 Operational Tooling, CI/CD & Quality Gates . . . 8

1 Executive Summary & Engineering Design Rationale

Modern web search applications typically force an uncomfortable architectural compromise: either rely on opaque third-party search APIs that monetize user telemetry, or deploy expensive stateful clusters (dedicated Elasticsearch nodes, headless browser fleets, and Redis caches) that cannot run on free-tier infrastructure.

Neutron v3.0 solves both challenges through a three-service **ConnectRPC/Protobuf** microservice topology backed by **Turso LibSQL FTS5** (with Zstandard full-document compression) and an on-demand **Just-In-Time (JIT)** live crawler. Rather than treating every query as an expensive scrape or relying exclusively on stale batch indexes, Neutron merges *cache-first sub-15 ms FTS5 retrieval* with *bounded-deadline parallel web discovery* and *cited synthesis*.

Why Neutron's Architecture Reflects Senior Systems Engineering

1. **True Microservice Isolation via Contract-First RPC:** Presentation (`frontend/`), edge coordination (`services/gateway`), search/synthesis (`services/search`), and outbound crawling (`services/crawler`) are isolated behind `proto/neutron/v1/neutron.proto`, communicating over HTTPS ConnectRPC (`pyqwest + certifi` root CAs) with constant-time `hmac.compare_digest` Bearer validation across both unary and server-streaming RPCs.
2. **Designed Explicitly for `0.1 vCPU` / `512 MB` Cold-Start Environments:** Every bottleneck—from `12 KB` Charikar SimHash truncation and `80`-link scan caps to `55 s` RPC timeouts, throttled downstream `/health` warm-up pings, and `8.5 s` bounded async JIT tasks—is tuned to prevent event-loop starvation and cold-start failures on Render Free web instances.
3. **Linguistically Aware Hybrid Retrieval:** Conversational questions (*"what is the main purpose of..."*, *"when was X established"*) are normalized, stripped of filler nouns, expanded across morphological

inflections into explicit FTS5 `AND` / `70%` quorum clauses, and re-ranked by title/URL/authoritative-domain boosts minus template boilerplate penalties.

4. **Zero-Telemetry Privacy by Construction:** Zero cookies, zero client IP persistence (rate limits use volatile in-memory `X-Forwarded-For` buckets), zero external icon CDNs, strict `DOMPurify` + `safeHttpUrl` sanitization in React, and signed 64-bit `url_hash` string serialization avoiding JavaScript IEEE-754 precision loss beyond $2^{53} - 1$.

2 Cloud Architecture & Deployment Topology

Neutron runs across three coordinated cloud planes: a static global edge UI on **GitHub Pages**, three Python 3.12.8 web services on **Render Free (Oregon)**, and an edge-replicated serverless database on **Turso LibSQL**. Figure 1 illustrates the complete network boundary, transport protocols, authentication flows, and external providers.

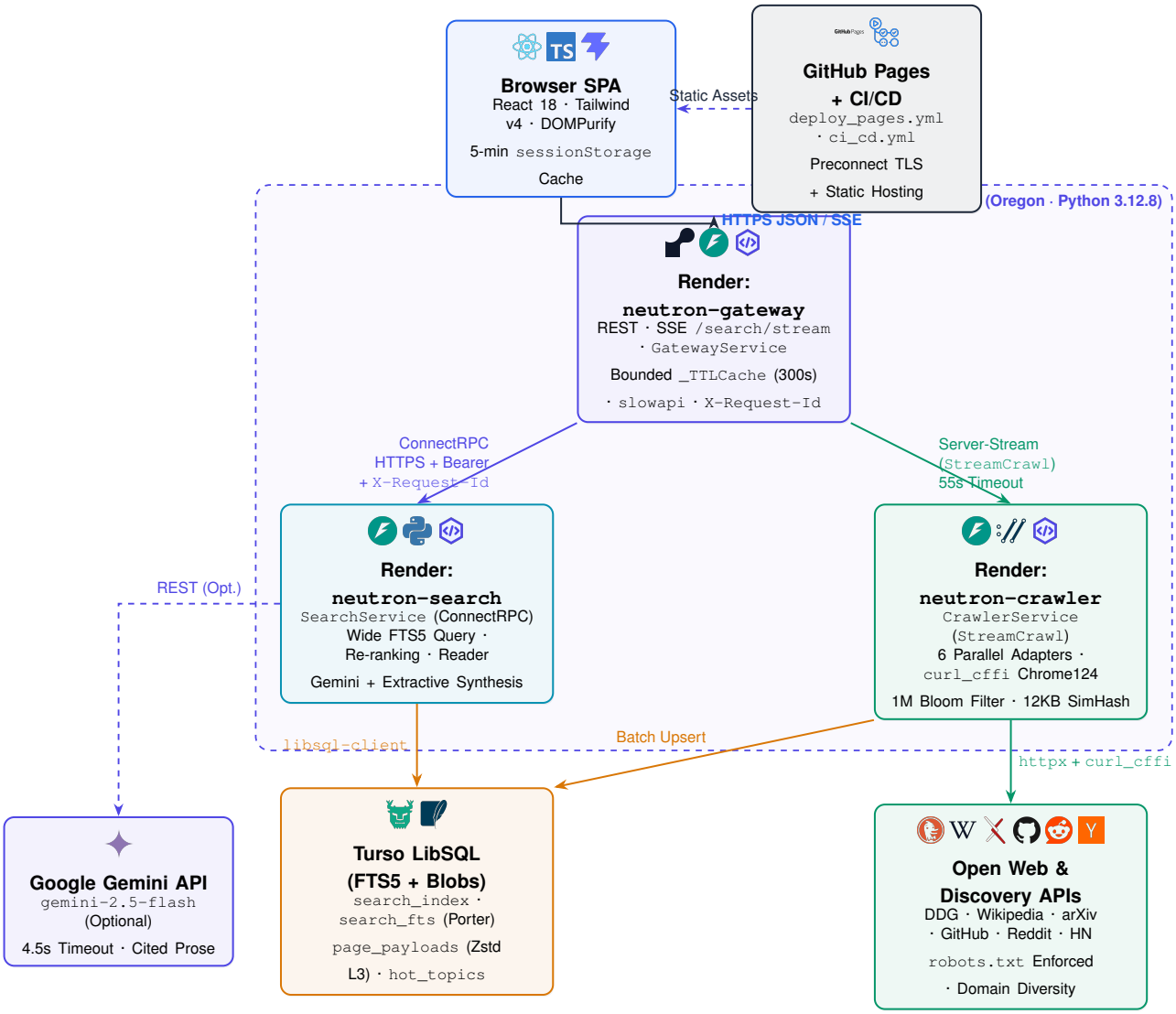


Figure 1: Neutron v3.0 Cloud Architecture showing actual edge, compute, storage, AI, and discovery providers.

2.1 Render Blueprint & Cold-Start Resilience Engineering

Render Free services automatically spin down after 15 minutes of inactivity and cannot accept private internal-network requests across free instances. Neutron's configuration (`render.yaml` and `services/common/config.py`) handles this transparently:

- **Public TLS ConnectRPC Transport:** `services/gateway/app.py` configures a shared `pyqwest.HTTPTransport` loaded with `certifi` root CA certificates plus system certs, authenticating calls to `https://neutron-search.onrender.com` and `https://neutron-crawler.onrender.com` via `INTERNAL_API_KEY` (auto-shared via Render `fromService`).
- **Dual Downstream Warm-Up (`_maybe_wake_downstream`):** Requests to `/health`, `/stats`, or downstream `502/503` recovery paths fire throttled non-blocking `/health` pings (at most once every `180 s`) to wake *both* `neutron-search` and `neutron-crawler` concurrently before search traffic arrives.
- **Graceful Cold-Start Envelopes:** `RPC_TIMEOUT_MS=55000` spans cold container boots (`30–50 s`). Early TCP/TLS handshakes are pre-warmed by `<link rel="preconnect">` in `frontend/index.html`. Custom `429` and `500` exception handlers ensure `CORSMiddleware` headers remain intact on error responses so the cross-origin GitHub Pages UI never fails opaquely.

3 High-Level Design (HLD): Hybrid Cache-First & JIT Streaming

Neutron's core architectural insight is avoiding an all-or-nothing choice between fast stale index lookups and slow live scraping. Controlled by `JIT_CACHE_THRESHOLD = 5`, every query flows through an adaptive multi-tier cascade (Figure 2).

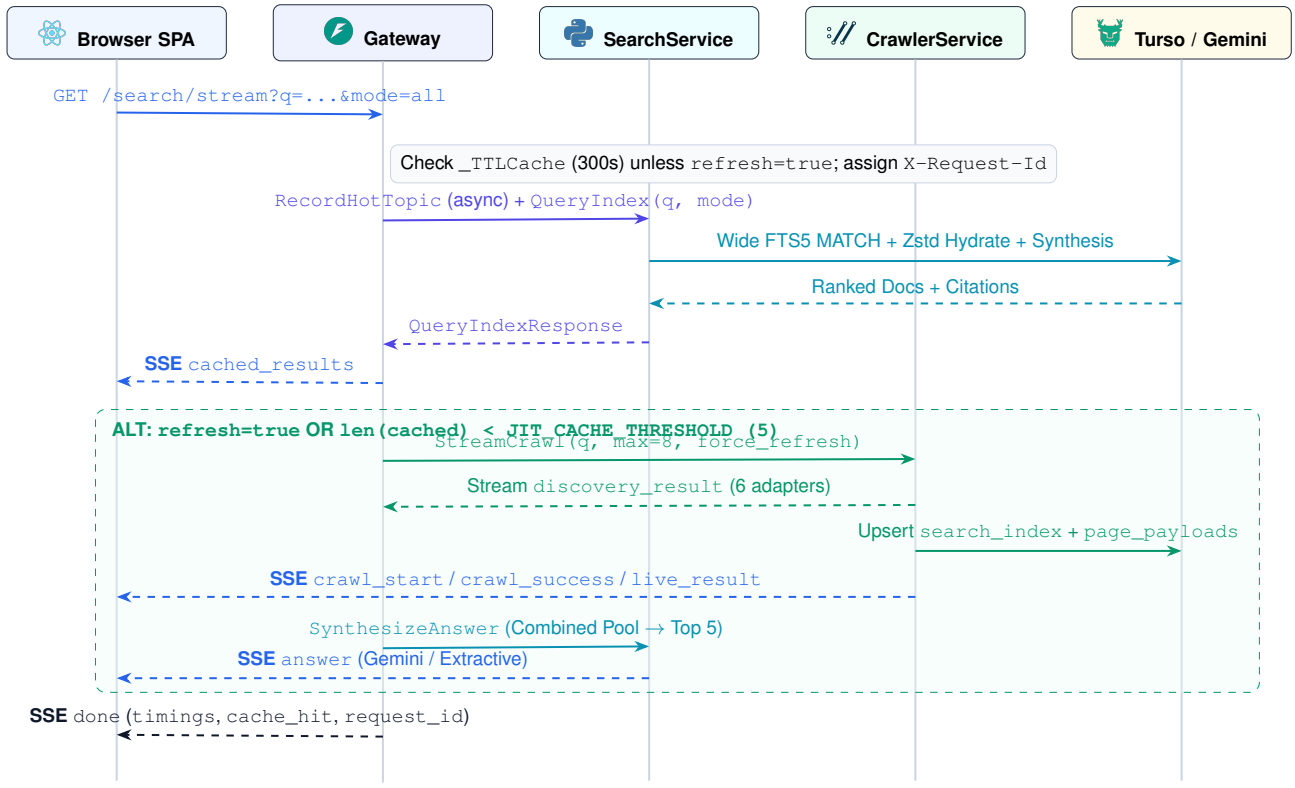


Figure 2: High-Level Design (HLD): Real-time SSE + ConnectRPC hybrid execution cascade.

3.1 Gateway Caching, Refresh Semantics & Error Resilience

The Edge Gateway (`services/gateway/app.py`) maintains a two-instance bounded LRU+TTL cache (`_TTLCache`): **256 entries at 300 s TTL** for `/search`, `/api/research`, and `/search/stream` keys, plus **30 s TTL** for connected `/stats` snapshots. When `?refresh=true` is requested from the UI Refresh action:

- Both Gateway's `_TTLCache` and Frontend's `5`-minute `sessionStorage` cache are bypassed.
- `StreamCrawl` and `DiscoverAndCrawl` receive empty `skip_urls` so previously indexed URLs can be re-fetched cleanly.
- Freshly crawled `SearchDocument` objects **replace stale cached entries in place** at matching URLs in both Gateway's merged payload and React's `live_result` state, and trigger re-synthesis over the combined pool (`max(12, limit)` candidates) so newly crawled facts elevate to the top.
- If both `SearchService` and `CrawlerService` fail (for instance, during a cold container restart), `/search/stream` emits an explicit `event: backend_error` and triggers downstream warm-up rather than falsely reporting zero matching results.

4 Low-Level Design (LLD): Internal Mechanics & Algorithms

4.1 Contract-First ConnectRPC, Auth Interceptor & 64-Bit IDs

All internal and edge RPC contracts reside in `proto/neutron/v1/neutron.proto` and compile via `scripts/gen_proto.py` (`protoc-gen-py + protoc-gen-connectrpc`) into `neutron_rpc/neutron/v1/`.

Service	RPC Methods (proto3)	Low-Level Responsibility
SearchService	`QueryIndex, SynthesizeAnswer, GetDocument, Suggest, GetTrending, GetStats, RecordHotTopic`	FTS5 stemmed query execution, wider vertical pre-fetch (`3x` for filtered modes), Zstd payload batch read, Gemini / extractive synthesis, autocomplete & trending.
CrawlerService	`DiscoverSeeds, CrawlUrl, DiscoverAndCrawl,` **`StreamCrawl (server stream)`**	Parallel multi-adapter URL discovery, `asyncio.Queue`-backed real-time crawl event streaming under `8.5 s` deadline, clean task cancellation on disconnect.
GatewayService	`Search, Suggest, GetStats, GetDocument`	ConnectRPC/gRPC-Web mirror alongside public REST/SSE endpoints on `neutron-gateway`, propagating `X-Request-Id` and Bearer credentials.

Table 1: Protobuf v1 service surface compiled into `neutron_rpc`.

Security Detail: Unified Unary + Streaming Auth & JS 64-Bit Precision

- **Streaming-Safe `InternalAuthInterceptor` (`services/common/rpc.py`):** Implements `connectrpc.interceptor.MetadataInterceptor[None]` (`on_start` / `on_end`) rather than unary-only interception, enforcing constant-time `hmac.compare_digest` checks on `Authorization: Bearer` / `x-internal-key` across both unary RPCs and server-streaming `StreamCrawl`. Hardcoded placeholder secrets are rejected at boot by Pydantic field validators.
- **String-Encoded Signed 64-Bit Hashes:** `hash_url()` and `hash_domain()` map

`xxhash.xxh64` digests to signed 64-bit SQLite integers $[-2^{63}, 2^{63}-1]$. Because JavaScript's `Number.MAX_SAFE_INTEGER` is $2^{53}-1$, Protobuf and JSON schemas transmit `url_hash` strictly as decimal strings to prevent reader lookup corruption.

4.2 Query Normalization, Stemmed Quorum FTS5 & Concept Re-Ranking

Natural-language questions severely degrade literal full-text recall if passed raw into SQLite FTS5. Figure 3 details Neutron's multi-stage lexical & semantic scoring pipeline (`spaces/crawler/db_turso.py` and `synthesis.py`).

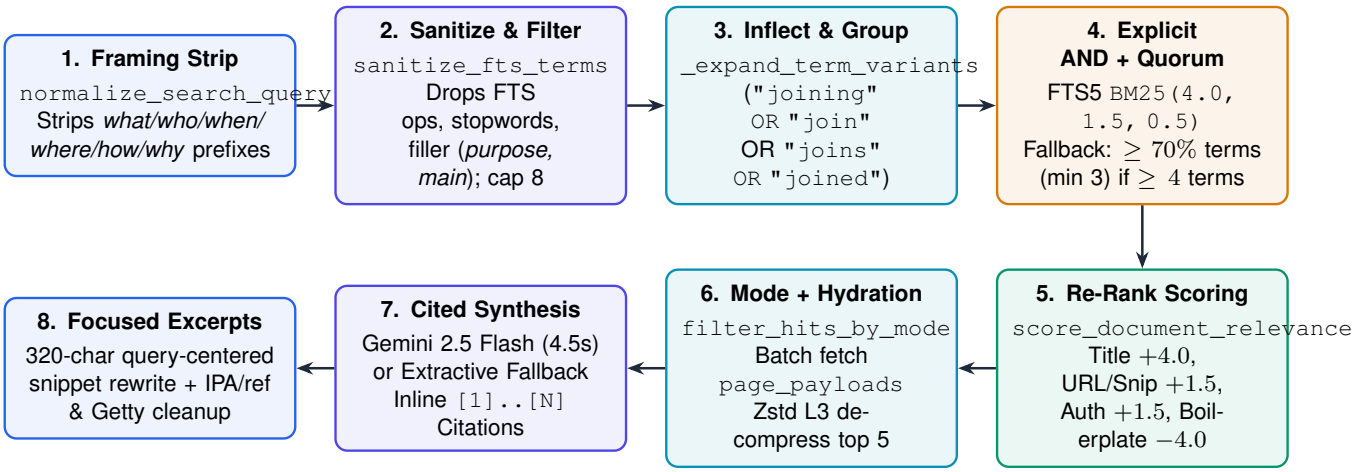


Figure 3: Low-Level Design (LLD): Query sanitization, SQLite FTS5 safe grouping, re-ranking, and synthesis pipeline.

For example, the conversational query *"what is the main purpose of the joining in SQL"* is reduced cleanly to `["joining", "sql"]`, expanded to `("joining" OR "join" OR "joins" OR "joined") AND "sql"`, retrieved via SQLite FTS5 Porter-stemmed BM25, and re-ranked using `(matched_terms, rel_score, db_score)` so authoritative SQL references outrank generic template repositories containing `main purpose of this project` (`-4.0` penalty when `mode != "code"`).

4.3 Discovery Scheduling, Chrome TLS Fetching & Deduplication

The Crawler microservice (`services/crawler/app.py`, `spaces/crawler/`) enforces strict politeness, diversity, and deduplication invariants:

1. **Parallel Discovery & 2-Pass Domain Diversity (`discovery.py`):** Concurrently queries active vertical adapters (DuckDuckGo HTML, Wikipedia API, HackerNews Algolia, Reddit JSON, arXiv Atom, GitHub Search, plus optional Brave Search) filtered by an active 24 h WAF cooldown map (`_get_active_waf_domains`). Pass 1 caps `1` URL per hostname for source diversity; Pass 2 backfills multi-article knowledge hubs (`en.wikipedia.org`, `arxiv.org`, `github.com`, `stackoverflow.com`) up to `max_urls`.
2. **Polite Fetch & Parsing (`robots.py`, `parser.py`):** Checks per-domain cached `robots.txt` (`NeutronBot/3.0`, 24 h TTL, `asyncio.Lock` per domain, fractional `Crawl-delay` parser) before executing `curl_cffi.requests.AsyncSession(impersonate="chrome124")` under an `asyncio.Semaphore(12)`. `selectolax.lexbor` extracts `__NEXT_DATA__` SSR state, JSON-LD author/publish timestamps, clean Markdown blocks, `max_links=80` followable outbound links (filtering `nofollow/sponsored/ugc`), and rejects Cloudflare/Akamai interstitials (`is_waf_or_empty_page`).

3. **Fast Bounded Deduplication (`bloom.py`, `dedup.py`, `compression.py`):**
   - *URL Bloom Filter:* 10^6 capacity (`0.1%` FP rate) using `mmh3` + `bitarray`, warmed from Turso `search_index` on crawler startup.
   - *12 KB SimHash:* 64-bit Charikar fingerprint over weighted unigrams and 2-gram shingles bounded to the leading `12,000` characters (`simhash_ms` reduced from `293 ms` to `15 ms`), checking per-domain `Hamming <= 3` before atomic write.
   - *Zstandard Payloads:* Compresses extracted Markdown via `zstd.ZstdCompressor(level=3)` and writes `domains`, `search_index`, and `page_payloads` in one Turso batch transaction.

5 Storage Schema & Data Modeling (Turso LibSQL FTS5)

All relational metadata, full-text virtual indexes, compressed page blobs, frontier queues, and trending topic counters coexist inside one serverless Turso LibSQL database (migrations/03_turso_fts5_schema.sql), eliminating multi-database sync overhead.

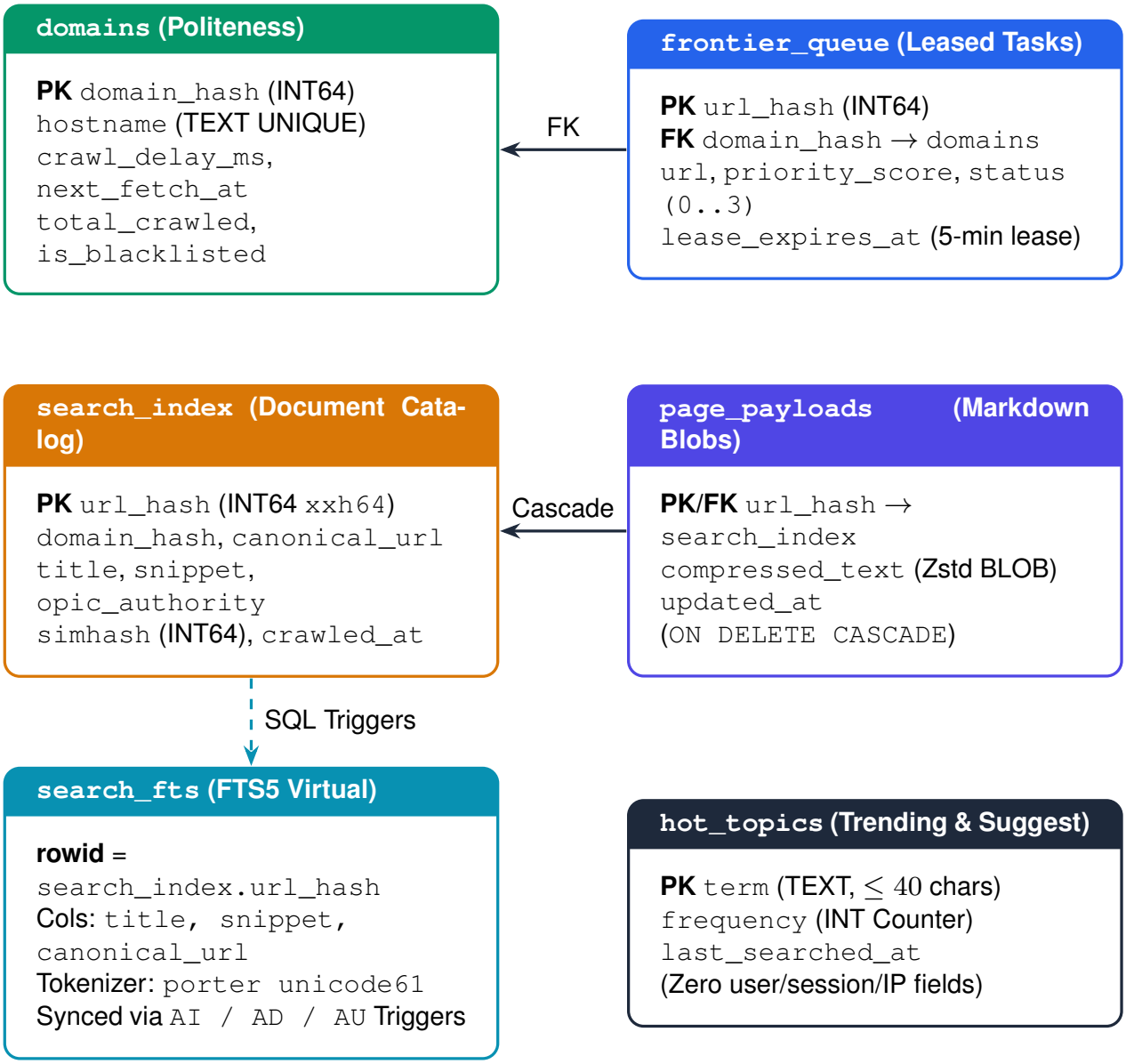


Figure 4: Turso LibSQL Entity-Relationship Diagram (migrations/03_turso_fts5_schema.sql).

6 Latency Budgets, Free-Tier Engineering & Verification

Operating Python async microservices on Render Free (0.1 vCPU, 512 MB RAM) requires strict deadline and CPU budgeting so long-tail targets never starve fast sources. Figure 5 summarizes the configured stage budgets across Neutron's execution paths.

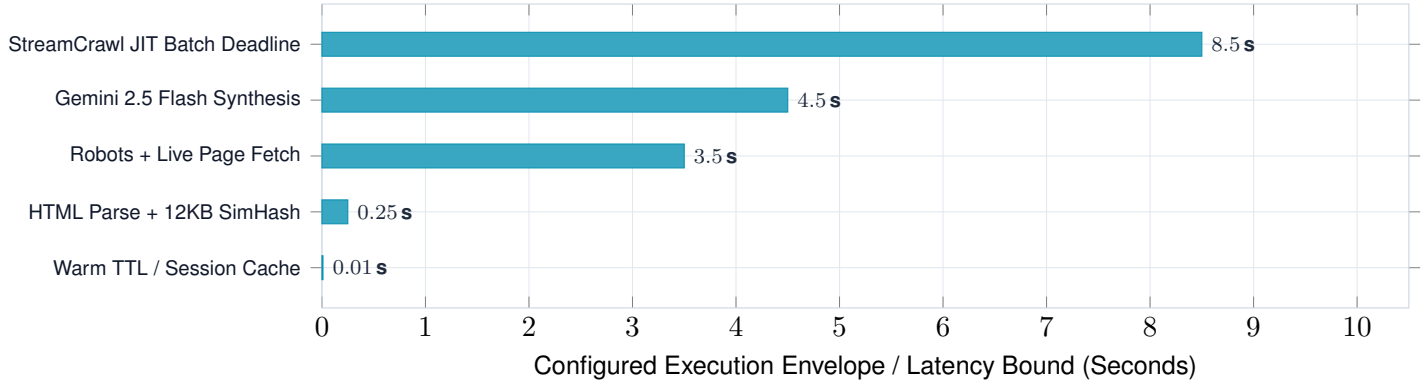


Figure 5: Configured latency ceilings (`services/common/config.py`) balancing recall and `0.1` vCPU responsiveness.

6.1 Operational Tooling, CI/CD & Quality Gates

Pipeline / Script	Verified Role in Current Stack
`.github/workflows/ci_cd.yml`	Runs `compileall`, `ruff check --select E9,F63,F7,F82,F401`, React `npm run build` (`tsc -b && vite build`), `44` unit + live ConnectRPC/SSE tests, and Turso migration on `main`.
`.github/workflows/deploy_pages.yml`	Compiles the Vite React SPA targeting `VITE_API_BASE_URL` (`https://neutron-gateway.onrender.com`) and publishes static bundles to GitHub Pages.
`scripts/gen_proto.py`	Recompiles `proto/neutron/v1/neutron.proto` into `neutron_rpc/neutron/v1/` ConnectRPC & Protobuf stubs.
`scripts/refresh_stale.py`	Queries oldest rows via `row.asdict()`, validates `robots.txt` + `text/html`, and updates `search_index` + Zstd `page_payloads` preserving `1,200`-char snippets.

Table 2: Verified CI/CD workflows and operational maintenance utilities.

Verified Privacy & Security Guarantee

Neutron stores **zero cookies**, **zero advertising IDs**, and **zero client IP addresses**. Rate limiting (`30/min` search, `15/min` stream, `20/min` research) uses ephemeral in-memory keys; Reader Markdown passes through `marked` and `DOMPurify` (`ALLOWED_URI_REGEXP: ^https?://`); and all external anchors validate protocols via `safeHttpUrl`.